

Untangling Scattered Compliance Logic

An SSOT Architecture for the Multi-Jurisdiction HR Compliance Automation Pipeline at Insperity

Architecture Proposal — Domain-Centric Metadata & Execution-Translation Pipeline, Applied to Federal / State / Local Employment Compliance

Prepared for internal architecture review · Insperity, Inc.

Grounded in Insperity's public profile as a professional employer organization (PEO) operating under a co-employment model, and in the specific statutes, PEO accreditation standards, and service-domain framework supplied for this review. Internal system names (payroll engine, benefits platform, HRIS) are illustrative placeholders; the compliance obligations they enforce — FLSA, ERISA, ACA, PWFA, OSHA, CPEO/ESAC/SOC, and the jurisdiction-specific statutes cited throughout — are real and should be reconfirmed against current law before this is used operationally.

Contents

Executive Summary

Why This Fits Insperity's PEO Model

The Problem: Compliance Logic Has No Fixed Address

1. Classify Logic by Layer — the “Right Home” Principle

2. Treat Logic as Code — the Governance Pipeline

3. The Modern Pattern: Declarative Spec + Generated Executables

4. Practical Migration Blueprint for Legacy Systems

5. PEO-Specific Governance Overlay: Accreditation & Attestation Standards

6. A Complementary View: Service-Domain Capability Lifecycle

Key Pitfalls to Avoid

Recommended Next Steps

Executive Summary

As a professional employer organization operating under a co-employment model, Insperity answers the same wage-and-hour, leave, tax-withholding, and benefits-eligibility questions differently depending on where each worksite employee physically works — across all 50 states, plus a growing layer of city and county ordinances (Los Angeles, San Francisco, New York City, Westchester County, and others) that impose stricter rules on top of state and federal baselines. That footprint means compliance logic at Insperity's scale is scattered across the payroll engine's parameter tables, the benefits administration platform's configuration, HRIS custom fields, and — inevitably — spreadsheets individual compliance analysts maintain by hand. There is no single place that says, definitively, which rule applies to which employee, in which jurisdiction, as of which effective date.

This proposal adapts a domain-centric Single Source of Truth (SSOT) architecture to that problem. Two ideas do the work:

- **A jurisdiction-centric Single Source of Truth.** Every constraint, rule, and computation is defined once, in a declarative spec, under version control — organized as Federal (Core), then State, then City/County, mirroring the actual legal hierarchy those rules already have.
- **An explicit execution-translation pipeline.** A CI/CD build step compiles that one spec into whatever each runtime needs — typed code for payroll and HRIS microservices, a jurisdiction relationship graph for cross-jurisdiction reasoning, or DMN/decision-table XML for the compliance rules engine — instead of forcing every system to speak the same language.

This is deliberately not a call to replace the existing payroll engine or benefits platform. It treats compliance logic as a governed, versioned data artifact, wraps the legacy estate behind a standardized Decision API, and lets each system keep doing what it already does well.

Two additions in this revision ground the architecture in Insperity's actual regulatory and operating footprint rather than a generic HR-compliance model: Section 5 adds the PEO-specific accreditation overlay (CPEO, ESAC, SOC 1/SOC 2, the Client Allocation Matrix) that sits above every jurisdiction's rule set, and Section 6 adds a second, complementary organizing view — the five PEO service domains (Benefits, Payroll & Tax, Compliance & Risk, Talent, HR Technology), each walked through a Discover → Define → Design → Develop → Deliver → Run capability lifecycle — cross-linked to the same jurisdiction-organized SSOT artifacts described in Sections 1–4.

Why This Fits Insperity's PEO Model

Insperity (NYSE: NSP), headquartered in Kingwood, near Houston, Texas, delivers HR and administrative services to small and mid-sized businesses through a co-employment arrangement. That business model is precisely why jurisdiction is the load-bearing axis for compliance logic here in a way it may not be for a single-location employer: the same Employee schema, the same FLSA overtime rule, and the same payroll tax engine have to correctly layer state- and city-specific variations on top of a shared federal baseline, for every client company, in every location that company has employees. A rule written once and misapplied across even a small fraction of Insperity's worksite-employee population is a wage-and-hour or leave-law exposure multiplied across every affected client — which is exactly the failure mode an SSOT with jurisdiction-aware versioning and CI is built to catch before it reaches a pay run.

The Problem: Compliance Logic Has No Fixed Address

Legacy HR/payroll estates accumulate three failure modes simultaneously, and they compound each other:

- **Duplication.** The same overtime rule lives as a parameter in the payroll engine, a config flag in the benefits platform, and a line in an HR generalist's desk-reference spreadsheet — each maintained independently, each one a chance to drift from the other two.
- **Silent drift.** A state passes a paid-sick-leave amendment; the payroll engine's accrual parameters get updated during an urgent fix, but the employee handbook language and the HRIS-driven new-hire notice do not — or vice versa.
- **No audit trail.** Payroll engine parameter tables and benefits platform configuration get edited directly in production ahead of a legislative deadline, with no PR, no review, and no version history — a serious liability given how directly wage-and-hour and leave-law violations expose both Insperity and its client companies to back-pay claims, PAGA-style litigation, and DOL audits.

The instinctive fix — pick one tool and force everything into it — reliably fails here too. Procedural tax-withholding math doesn't belong in a rules engine, and a jurisdiction's legal precedence relationships don't belong hardcoded in application code. Section 1 gives each kind of compliance logic its correct home before anything is centralized.

1. Classify Logic by Layer — the “Right Home” Principle

Before centralizing anything, classify what kind of logic is in front of you. Mixing these categories — running tax-withholding arithmetic inside a rules engine, or hand-coding a state's meal-break premium as an if-statement buried in the payroll run orchestrator — is the single largest driver of legacy bloat in a multi-jurisdiction HR system.

Classify Logic by Layer — The “Right Home” Principle

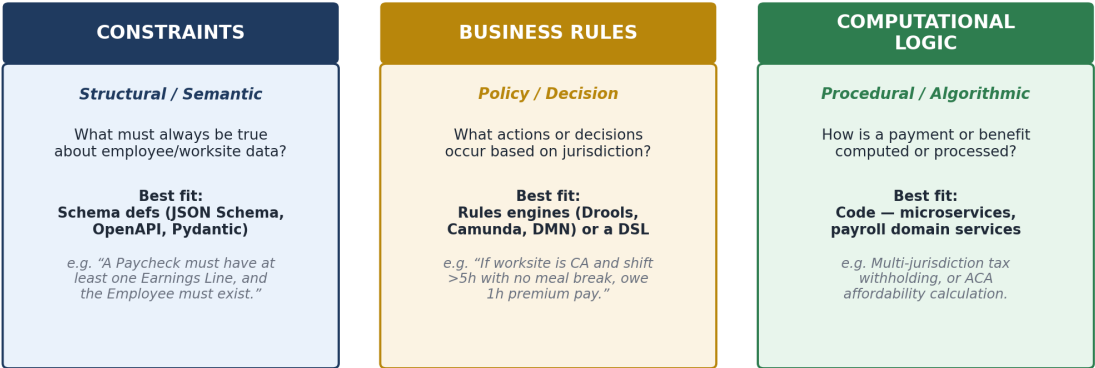


Figure 1. Three logic categories, each routed to the tool built for its paradigm.

Constraints (structural / semantic)	What must always be true about the domain data?	Ontologies (OWL/RDF); schema definitions (JSON Schema, OpenAPI, Pydantic, Protobuf)	An ERISA Master Plan Document must carry a Summary Plan Description and Form 5500 filing fields before it can be attached to a client's benefit offering; every EEO-protected-class value must resolve against the canonical Title VII, ADEA, ADA, and GINA taxonomy.
--	---	---	---

Business Rules (<i>policy / decision</i>)	What actions or decisions occur based on jurisdiction and state?	Rules engines (Drools, Camunda, DMN) or a DSL	If worksite is California and shift >5h with no meal break, owe 1h of premium pay; if a worker fails the federal economic-reality test <i>or</i> the stricter state ABC test where one applies, classify as Employee, not Independent Contractor.
Computational Logic (<i>procedural / algorithmic</i>)	How is a payment or benefit computed or processed?	Code — microservices, payroll/benefits domain services	Multi-state SUTA experience-rate allocation and local tax-withholding jurisdiction resolution; ACA Applicable Large Employer determination via the look-back measurement method.

2. Treat Logic as Code — the Governance Pipeline

Once logic is classified, it needs a Single Source of Truth: a central repository holding jurisdiction specs in human-readable form (YAML, DMN, Turtle/OWL), organized to mirror the legal hierarchy itself — Federal, then State, then City/County. The hard rule that makes this work: **nobody edits a compliance rule directly in the payroll engine's parameter UI, or in the benefits platform's admin console, in production.** Everything is checked into source control first, and reaches production only by passing through CI.

Continuous Integration for Compliance Logic

- **Linting & validation.** Semantic reasoners catch contradictory jurisdiction-precedence axioms — e.g. a proposed San Francisco rule that would silently contradict, rather than extend, the California state rule it's supposed to layer on top of. Static analysis catches malformed or conflicting DMN decision tables before merge.
- **Automated scenario testing.** Every rule or schema change runs against synthetic payroll runs pre-deployment. Given the litigation exposure of wage-and-hour and leave-law mistakes, this is the difference between a bad rule change getting caught in staging versus surfacing in a DOL audit or a class action.
- **Semantic versioning.** Jurisdiction rule sets and schemas are versioned explicitly. The payroll run orchestrator and every other consuming microservice declare which version of a jurisdiction's ruleset they depend on, so a mid-quarter rule change can't silently break a pay cycle already in flight.

The GitOps Loop for Compliance Logic

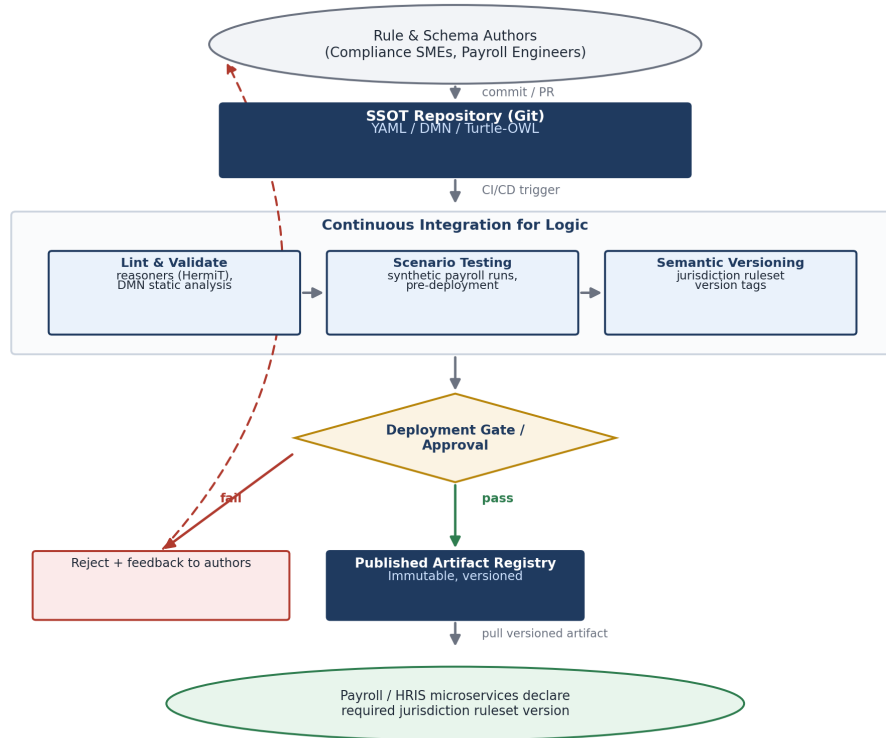


Figure 2. The GitOps loop for compliance logic: commit → CI validation → gated release → versioned consumption, with rejected changes routed back to the author.

3. The Modern Pattern: Declarative Spec + Generated Executables

Rather than rewriting the payroll engine, the benefits platform, and the HRIS to “speak the same language,” the architecture is declarative-first: jurisdiction logic is written once, and a build pipeline compiles it down to whatever each target runtime needs.

How it works

- **Write once, declaratively.** Employee/Worksite/Paycheck schema constraints and jurisdiction taxonomy are defined in OpenAPI/JSON Schema; compliance rules are defined in DMN.
- **Compile for runtimes.** Code generators in CI/CD produce, on every merge: TypeScript/Python interfaces for payroll and HRIS microservices, SHACL/OWL files for the jurisdiction relationship graph (which local ordinances extend which state law, which states share tax-withholding reciprocity), and DMN/Drools XML for the compliance rules-engine runtime.

Because all three artifacts regenerate from the same versioned spec on every build, they cannot drift out of sync with each other — drift becomes a build failure, not a payroll-run incident.

One Declarative Spec, Three Generated Runtime Artifacts

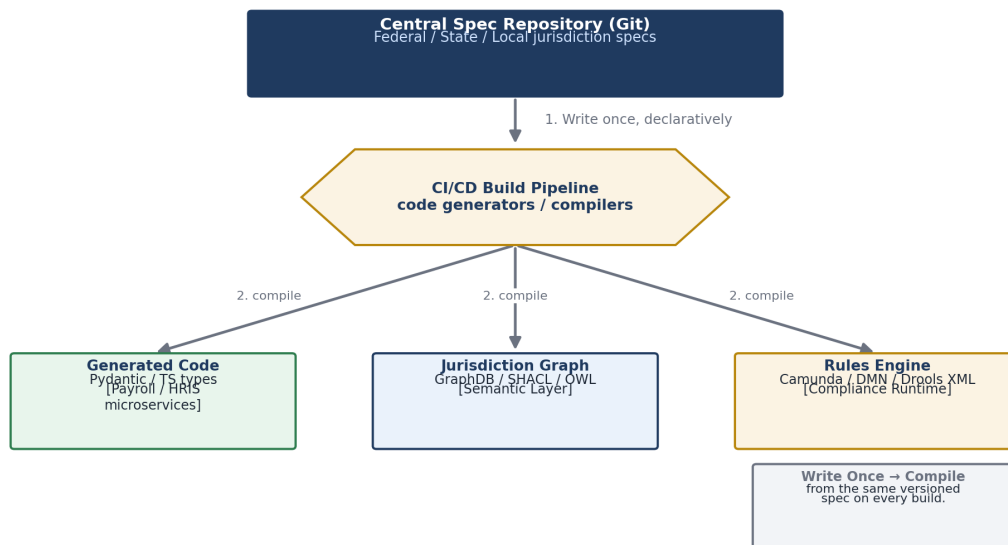


Figure 3. One declarative spec, three generated runtime artifacts, one CI/CD compiler in between.

Runtime View: The Decision API

At request time, callers — the payroll run orchestrator, a new-hire onboarding service, a scheduling app checking predictive-scheduling compliance — don’t talk to the rules engine, the jurisdiction graph, or a legacy payroll stored procedure directly. They talk to a standardized Decision API that resolves the worksite’s jurisdiction, resolves the caller’s declared ruleset version, validates the request against the

current schema, and routes to whichever backend currently owns that decision — including a wrapped legacy payroll engine that hasn't been migrated yet.

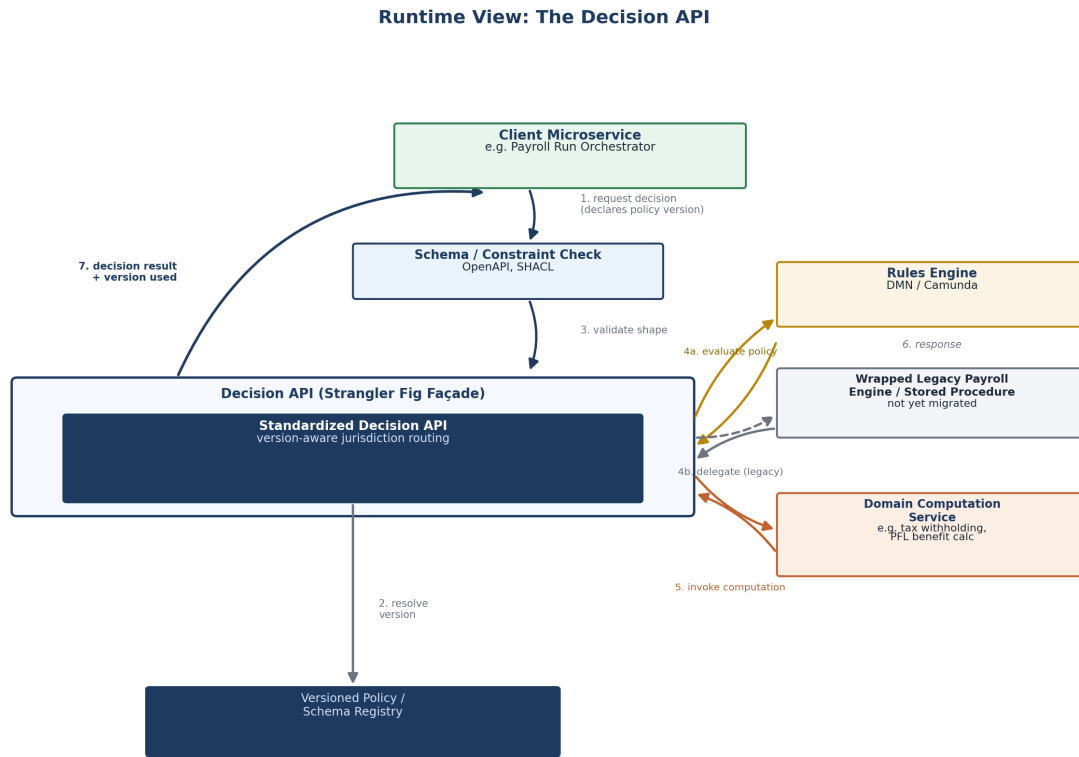


Figure 4. A client microservice never calls the rules engine, jurisdiction graph, or legacy payroll procedure directly — it calls the Decision API, which is free to route to legacy or modern backends without the client knowing which.

This indirection is what makes the migration in Section 4 possible: the façade lets legacy and modernized compliance logic sit behind one interface, so callers never need to change when the backend behind that interface changes.

A Note on an Alternative Architecture Considered

A more advanced pattern was also reviewed for this engagement: a holonic semantic-governance architecture in which rules are authored as LKIF deontic triples validated against W3C SHACL shapes, compiled through a multi-stage pipeline into OPA/Rego policy bundles and dbt/SQL, and reconciled across jurisdictions via SPARQL-based “most-protective-rule” harmonization — with dedicated guardrails for AI agents acting on the rule set. That pattern is a legitimate direction for a compliance platform at significant scale and rule-authoring velocity, and nothing here forecloses it.

It was deliberately not adopted as the starting architecture. The declarative-DMN-plus-Decision-API pattern in this proposal solves the same core problem — one governed source of truth, generated consistently into every runtime — with dramatically less new infrastructure to build, staff, and operate before the first jurisdiction rule ships, and it maps directly onto tooling (DMN decision tables, JSON Schema, a REST façade) that engineering and compliance SMEs can review without first learning a semantic-web stack. The SHACL/OPA-style approach is a coherent *next step*, not a competing starting

point: once the SSOT is established and jurisdiction coverage is broad enough that cross-jurisdiction “most protective rule” reasoning becomes a recurring, expensive manual exercise, the ontology/graph store already reserved in this architecture (Section 3, “How it works”) is the natural place to introduce SHACL shape validation and OPA-based policy compilation incrementally, without a rearchitecture.

4. Practical Migration Blueprint for Legacy Systems

For compliance logic trapped in payroll engine parameter tables, HRIS custom fields, and compliance-analyst spreadsheets, migrate in three phases rather than attempting a rewrite:

Phase 1 — Logic Discovery & Extraction

- **Inventory & audit.** Trace payroll runs and audit configuration across the payroll engine, benefits platform, and HRIS. Log every point where a compliance decision is currently made.
- **Classify.** Sort each finding into Constraint, Rule, or Computation using the framework in Section 1.

Phase 2 — Decouple via API Boundaries (Strangler Fig)

- **Wrap, don't rewrite.** Put the legacy payroll engine and benefits platform behind the standardized Decision API from Section 3 before touching their internals.
- **Extract incrementally.** Pull hardcoded jurisdiction logic out of application code into DMN or config, one ordinance at a time, re-pointing the façade as each piece lands.

Phase 3 — Centralize Schemas First

- **Standardize data shapes.** Unify Employee, Worksite, and Paycheck definitions across every state and local jurisdiction extension via a common schema registry, before consolidating rules.
- **Consolidate rules second.** Once the underlying data shapes are unified, reconciling jurisdiction rules becomes dramatically simpler — most apparent conflicts between, say, California and New York sick-leave rules turn out to be differences in how “covered employee” or “accrual period” is defined, not genuine policy disagreements.

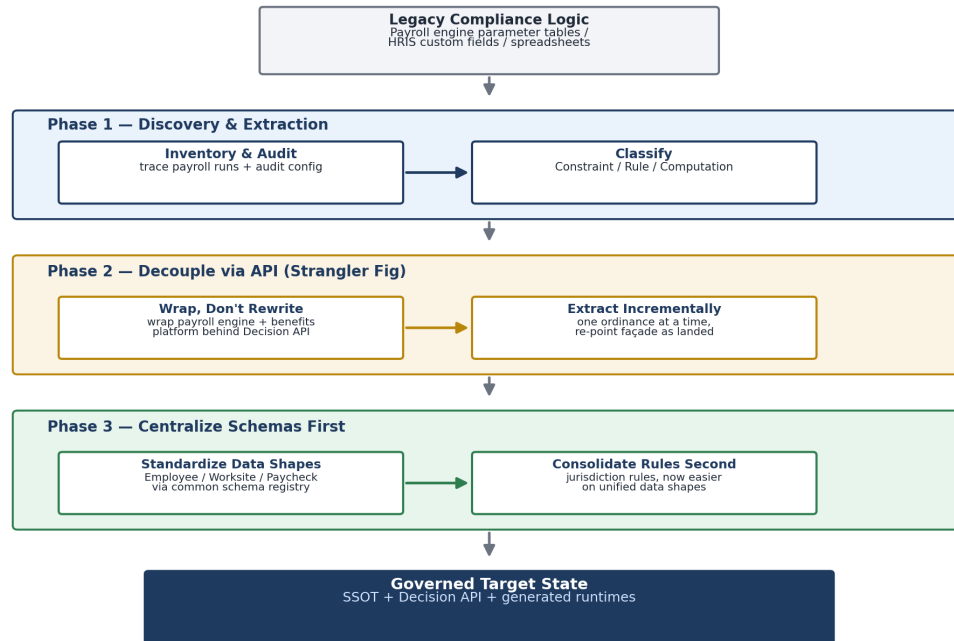
Three-Phase Strangler-Fig Migration Blueprint

Figure 5. Three-phase strangler-fig migration: discover and classify, decouple behind a Decision API, then centralize schemas before consolidating jurisdiction rules.

5. PEO-Specific Governance Overlay: Accreditation & Attestation Standards

Everything in Sections 1–4 organizes compliance logic by jurisdiction — Federal, then State, then City/County. Operating as a PEO adds a second, cross-cutting layer that sits *above* every jurisdiction: a set of accreditation and attestation standards that exist specifically because Insperity is transmitting statutory liability relief and administering client-company employment obligations at scale, not just complying with the law on its own behalf.

Standard Framework /	Regulatory Body / Sponsor	Core Operational Purpose
IRS Certified PEO (CPEO)	Internal Revenue Service (IRC § 7705)	Transmits statutory sole-liability relief to clients for federal employment taxes; requires annual independent CPA audits, quarterly tax attestations, and federal bonding.
ESAC Accreditation	Employer Services Assurance Corporation	Functions as an industry-specific financial and operational assurance layer, with continuous monitoring and client protection bonding, comparable in role to FDIC oversight of depository institutions.
SOC 1 (Type II) / SOC 2	AICPA (SSAE 18 / ISAE 3402)	Validates internal controls over payroll-processing financial reporting, and over data privacy, cloud-architecture security, and multi-tenant isolation, respectively.
Client Allocation Matrix	State PEO Licensing Boards / Model PEO Acts	Formalizes the division of statutory responsibilities and liability limits between the PEO and the client company within the Master Services Agreement.

These standards don't get their own jurisdiction node in the SSOT tree — they're constraints that apply to the governance *process itself*. Concretely, that means: the CPEO Certification Constraint gates which artifacts are eligible to carry sole-liability-relief semantics; ESAC and SOC 1/SOC 2 attestation evidence is generated as a by-product of the same CI audit trail (Section 2) that already records every reviewer, approval, and release; and the Client Allocation Matrix is the authoritative source for which party — Insperity or the client company — a given rule's obligation ultimately binds, which is exactly the kind of fact a Decision API caller (Section 3) needs resolved before it can act on a rule's output.

6. A Complementary View: Service-Domain Capability Lifecycle

The jurisdiction axis answers “which rule applies, where.” A second, independent axis answers a different question that Insperity's own service organization is structured around: “which of our five PEO service domains owns this capability, and how mature is it?” Every domain — Employee Benefits, Payroll & Tax Administration, HR Compliance & Risk Management, Talent Management & Development, and HR Technology — is walked through the same six-phase capability lifecycle: Discover → Define → Design → Develop → Deliver → Run.

Service Domain	Representative scope across Discover → Run	Sample linked SSOT artifacts
Employee Benefits	ACA ALE determination and affordability safe harbors; ERISA MEP/PEP plan governance and SECURE 2.0 LTPT eligibility; annual NDT testing and Form 5500 / 1094-C / 1095-C filings.	<i>ACA ALE Determination Rule, ERISA Plan Governance, SECURE 2.0 LTPT Eligibility Rule, COBRA Qualifying Event Rule</i>
Payroll & Tax Administration	Multi-state tax nexus and SUTA experience-rate allocation; FLSA salary-basis thresholds and overtime-engine logic; quarterly CPEO attestations and Form 941/W-2 filings.	<i>Multi-State Tax Nexus Engine, Worker Classification Test, FLSA Overtime Eligibility Rule</i>
HR Compliance & Risk Management	Workers'-comp and OSHA hazard baselines; CA SB 553 workplace-violence and Cal/OSHA heat-illness plans; PWFA and PUMP Act accommodation workflows; OSHA Form 300/300A/301 electronic reporting.	<i>EEO Protected Class, PWFA Accommodation Rule, PUMP Act Lactation Rule, OSHA Electronic Recordkeeping Rule, Workplace Violence Prevention Rule, Heat Illness Prevention Rule</i>
Talent Management & Development	EEOC selection-procedure guidelines and state pay-transparency laws; AEDT bias-audit and candidate-notice statutes (e.g. NYC Local Law 144); FCRA background-check sequencing; EEO-1 and state pay-data reporting.	<i>AEDT Bias Audit Rule, NYC Local Law 144 Notice Rule, FCRA Background Check Workflow, Sexual Harassment Prevention Training Rule</i>

Service Domain	Representative scope across Discover → Run	Sample linked SSOT artifacts
HR Technology	Multi-tenant data sovereignty and the PEO Client Allocation Matrix; SOC 1 / SOC 2 and HIPAA Security Rule controls; state privacy statutes (CCPA/CPRA) and role-based access control.	<i>Employee, Worksite (canonical Core entities underlying every domain above)</i>

This is not a rival organizing principle to the jurisdiction-centric SSOT — it's a second index into the same underlying artifacts. A given released rule, e.g. the AEDT Bias Audit Rule, is simultaneously “a Federal-namespaces Rule artifact, extended by NYC's Local Law 144 Notice Rule” (the jurisdiction view) and “a Talent Management & Development capability, currently in the Design/Develop phase of its lifecycle” (the service-domain view). The dashboard companion to this proposal exposes both views over one shared artifact set, so a compliance analyst can navigate by jurisdiction and a service-line owner can navigate by domain without maintaining two separate systems of record.

Key Pitfalls to Avoid

Pitfall	Why it bites, and how the SSOT prevents it
Treating a local ordinance as a global override	Local rules (LA, SF, NYC, Westchester...) must explicitly extend the state and federal baseline they layer onto, never replace it wholesale — otherwise an employee whose worksite changes silently loses or gains protections the ordinance never intended.
Skipping cross-jurisdiction testing in CI	A schema or rule change that passes CA-only scenario tests can still break the SF or LA ordinance layered on top of it. The scenario-test suite has to include the state <i>and</i> every local ordinance that extends it, not just the jurisdiction being edited.
Letting SMEs bypass the pipeline for urgent deadlines	The first “just this once” edit made directly in the payroll engine's parameter UI ahead of a legislative deadline becomes the template for the next one. Audit-trail integrity is itself a deliverable in a DOL or state labor-agency audit — it can't have gaps.
Modeling law changes as simple field edits, not dated rule sets	Wage-and-hour and leave-law changes routinely have future effective dates announced months ahead, and some are applied retroactively. The SSOT needs first-class support for “not yet in effect” and “effective as of” states — not just Draft / Candidate / Released.

Recommended Next Steps

- **Stand up the SSOT repository and CI pipeline first** — scoped initially to Federal plus one pilot state — before extracting a single rule. Governance has to exist before there's anything worth governing.
- **Pilot on one bounded decision** — California meal-and-rest-break premium pay is a good candidate — to prove the classify → spec → compile → deploy loop end-to-end, including the Decision API façade sitting in front of the existing payroll engine.
- **Inventory before you migrate.** Run Phase 1 across the full legacy estate — payroll engine, benefits platform, HRIS, and the spreadsheets compliance analysts maintain by hand — before committing engineering time to Phase 2 extraction. A PEO's 50-state-plus-local-ordinance footprint is larger than a single-location enterprise's, and classification will materially change the shape of the migration plan.
- **Make jurisdiction-version declaration mandatory from day one** for any service consuming a compliance ruleset. Given the litigation exposure of wage-and-hour and leave-law violations, retrofitting version discipline later is the most expensive place to accumulate debt.

A note on scope: the internal system names used throughout this proposal (payroll engine, benefits platform, HRIS) are illustrative placeholders standing in for Insperity's actual platforms. The statutes, PEO accreditation standards, and service-domain framework cited — FLSA, ERISA, ACA, PWFA, PUMP Act, OSHA, CPEO/ESAC/SOC, and the jurisdiction-specific rules across California, Los Angeles, San Francisco, New York State, New York City, and Westchester County — reflect real, current legal requirements as researched for this proposal, but statutory and regulatory detail changes frequently. Before this architecture is used operationally, Insperity's engineering and legal/compliance teams should reconfirm every citation against current law, substitute the real system names, and validate the pilot decision (Recommended Next Steps, above) against actual payroll-run data.